

青 / 春 / 不 / 老 / 梦 / 想 / 永 / 在

FEEL THE MEANING OF THE TRIP

DREAM

MY DREAM WILL NEVER STOP

计算思维与实践

实验11&12 链表与文件



哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

GO!
TAKE YOU ON A TRIP

探索 从未停止

目录

CONTENT

01

实验目的

02

实验内容

03

实验报告

04

期末检查



实验目的

- **掌握单链表和双链表的基本原理及其实现方法；**
 - **熟练实现链表的常见操作，包括节点的查找、插入与删除；**
 - **掌握通过文件完成链表数据持久化存储的方法。**
-

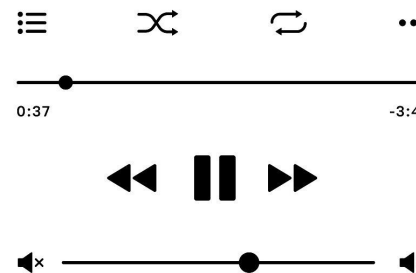
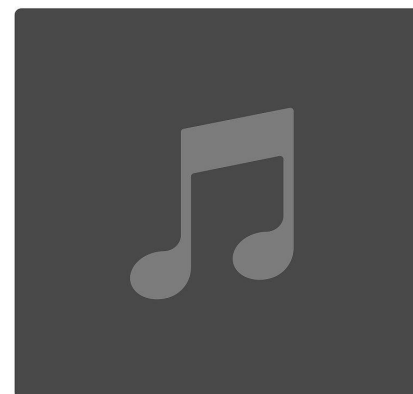


实验内容

实验11 音乐播放器 I

请你设计一个基于**单向链表**的音乐播放器歌单管理系统，实现以下功能：

- 从**文件**读取歌曲信息，并建立歌单链表
- 用户**添加**或**删除**歌曲信息
- 用户通过歌曲名**查找**并播放歌曲
- **显示**歌单列表
- 将更新后的歌单信息写回到**文件**





实验内容

//歌曲节点结构题

```
typedef struct Song{  
    int id;           //歌曲唯一标识  
    char title[100];  //歌名  
    char artist[50];  //歌手名  
    char filepath[300]; //音频文件路径  
    struct Song* next; //指向下一首歌的指针  
}Song;
```

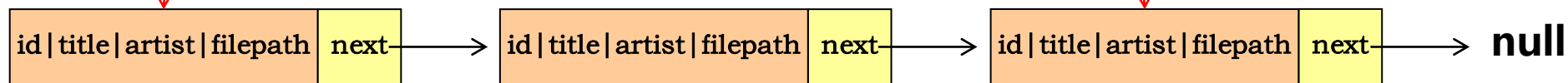
// 播放列表管理器结构体

```
typedef struct PlaylistManager{  
    Song* head;      // 链表头  
    Song* tail;      // 链表尾  
    Song* current;   // 当前播放/操作的歌曲  
    int song_count;  // 歌曲总数  
} PlaylistManager;
```

PlaylistManager



Song





实验内容

【输入描述】

程序启动时，从名为`song_list.txt`的文件中读取歌曲信息，创建链表。**文件中的每一行包含一首歌曲和对应的信息。**

用户选择添加歌曲功能，接下来从命令行读入歌曲信息，**插入**歌单**尾部**；

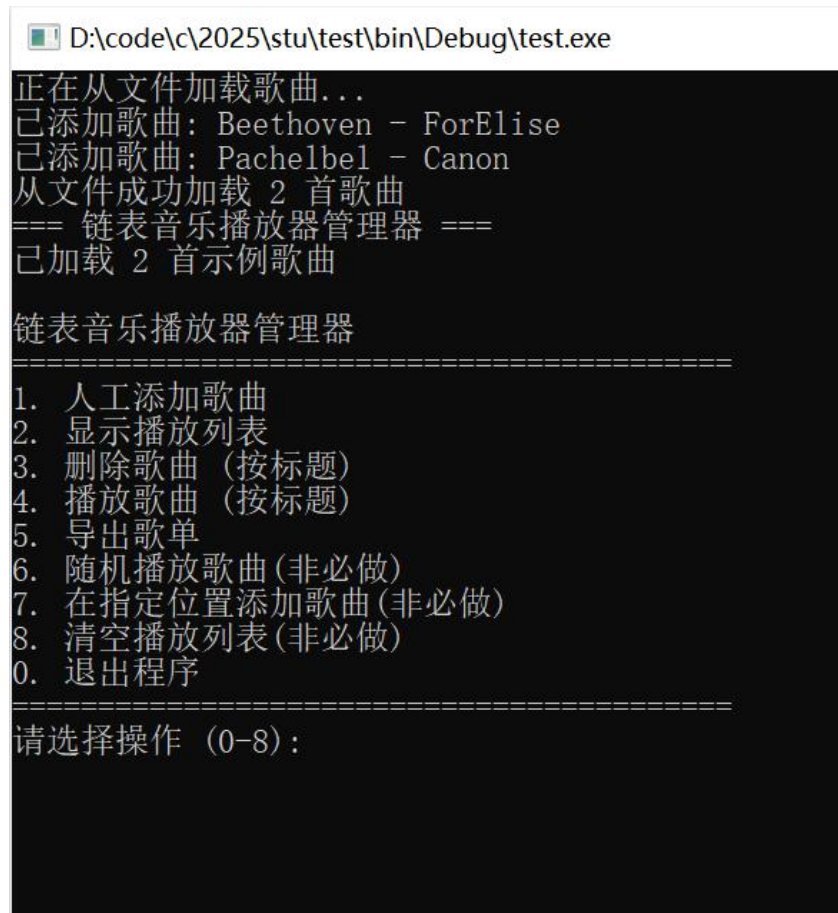
用户选择删除歌曲功能，接下来从命令行读入歌曲名，从歌单中**删除**歌曲；

用户选择播放歌曲功能，接下来从命令行读入歌曲名，**播放**该歌曲；

用户选择导出歌单功能，接下来导出歌单到**文件**中。

【输出描述】

把插入、删除操作结束后的歌单信息写回文件`song_list_result.txt`





实验内容

TODO:

//从文件中读取歌曲信息到链表

```
int load_songs_from_file(PlaylistManager* manager, const char* filename);
```

//在链表末尾添加歌曲

```
void add_song(PlaylistManager* manager, const char* title, const char* artist,  
              const char* filepath);
```

//显示播放列表

```
void display_playlist(PlaylistManager* manager);
```

//删除歌曲

```
int delete_songs_by_title(PlaylistManager* manager, const char* title);
```

//播放歌曲

```
int play_song_by_title(PlaylistManager* manager, const char* title);
```

//将播放列表保存到文件

```
int export_playlist(PlaylistManager* manager, const char* filename);
```

//随机播放歌曲 (非必做)

```
int play_song_random(PlaylistManager* manager);
```

//在指定位置插入歌曲 (非必做)

```
int insert_song_at(PlaylistManager* manager, int position, const char* title,  
                  const char* artist, const char* filepath);
```

实验内容

// 歌曲播放操作 (Windows)

```
void play_audio_windows(const char* filename) {  
    char command[256];  
    snprintf(command, "start \"%\" \"%s\"", filename);  
    ret = system(command);  
    if (ret != 0) {  
        printf("播放失败, 请检查文件是否存在或格式是否支持。\\n");  
    }  
}
```

```
D:\code\c\2025\stu\test\bin\Debug\test.exe  
正在从文件加载歌曲...  
已添加歌曲: Beethoven - ForElise  
已添加歌曲: Pachelbel - Canon  
从文件成功加载 2 首歌曲  
=== 链表音乐播放器管理器 ===  
已加载 2 首示例歌曲  
  
链表音乐播放器管理器  
=====
```

1. 人工添加歌曲		
2. 显示播放列表		
3. 删除歌曲 (按标题)		
4. 播放歌曲 (按标题)		
5. 导出歌单		
6. 随机播放歌曲 (非必做)		
7. 在指定位置添加歌曲 (非必做)		
8. 清空播放列表 (非必做)		
0. 退出程序		

```
=====
```

请选择操作 (0-8): 2

? 播放列表 (2 首歌曲)

```
=====
```

1. ForElise	- Beethoven	(musics/ForElise.mp3)
2. Canon	- Pachelbel	(musics/Canon.mp3)

```
=====
```

按回车键继续...

链表音乐播放器管理器

```
=====
```

1. 人工添加歌曲		
2. 显示播放列表		
3. 删除歌曲 (按标题)		
4. 播放歌曲 (按标题)		
5. 导出歌单		
6. 随机播放歌曲 (非必做)		
7. 在指定位置添加歌曲 (非必做)		
8. 清空播放列表 (非必做)		
0. 退出程序		

```
=====
```

请选择操作 (0-8): 1

请输入歌曲标题: 小美满

请输入作者: 周深

请输入歌曲路径: musics/小美满.mp3

已添加歌曲: 周深 - 小美满

按回车键继续...

```
D:\code\c\2025\stu\test\bin\Debug\test.exe  
0. 退出程序  
=====
```

请选择操作 (0-8): 2

? 播放列表 (3 首歌曲)

```
=====
```

1. ForElise	- Beethoven	(musics/ForElise.mp3)
2. Canon	- Pachelbel	(musics/Canon.mp3)
3. 小美满	- 周深	(musics/小美满.mp3)

```
=====
```

按回车键继续... 4

链表音乐播放器管理器

```
=====
```

1. 人工添加歌曲		
2. 显示播放列表		
3. 删除歌曲 (按标题)		
4. 播放歌曲 (按标题)		
5. 导出歌单		
6. 随机播放歌曲 (非必做)		
7. 在指定位置添加歌曲 (非必做)		
8. 清空播放列表 (非必做)		
0. 退出程序		

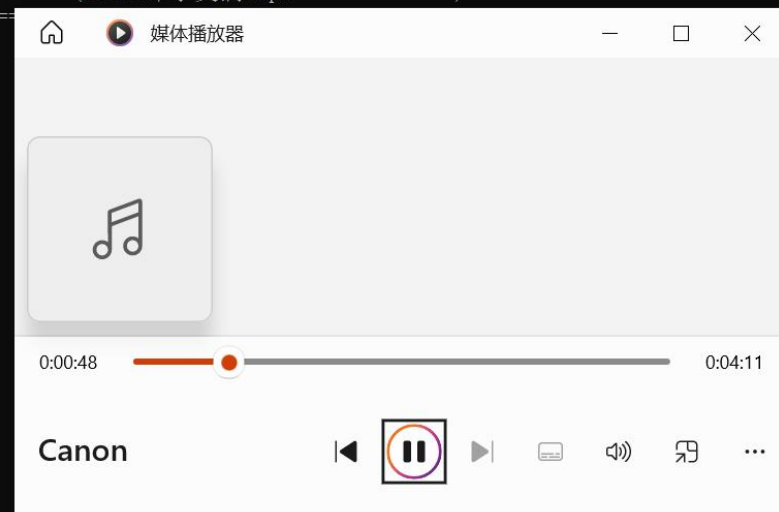
```
=====
```

请选择操作 (0-8): 4

请输入要播放的歌曲标题: Canon

Found File!!已播放标题为“Canon”的歌曲

按回车键继续...





实验内容

D:\code\c\2025\stu\test\bin\Debug\test.exe

链表音乐播放器管理器

1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲 (非必做)
7. 在指定位置添加歌曲 (非必做)
8. 清空播放列表 (非必做)
0. 退出程序

请选择操作 (0-8): 3

请输入要删除的歌曲标题: ForElise

已删除歌曲: Beethoven - ForElise (path: musics/ForElise.mp3)

共删除 1 首标题为 "ForElise" 的歌曲

按回车键继续...

链表音乐播放器管理器

1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲 (非必做)
7. 在指定位置添加歌曲 (非必做)
8. 清空播放列表 (非必做)
0. 退出程序

请选择操作 (0-8): 2

? 播放列表 (2 首歌曲)

- | | | |
|----------|-------------|-------------------|
| 1. Canon | - Pachelbel | (musics/Canon.mp3 |
| 2. 小美满 | - 周深 | (musics/小美满.mp3 |

链表音乐播放器管理器

1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲 (非必做)
7. 在指定位置添加歌曲 (非必做)
8. 清空播放列表 (非必做)
0. 退出程序

请选择操作 (0-8): 5

请输入要导出的目标文件名: song_list_result.txt

已保存 2 首歌曲到文件: song_list_result.txt

按回车键继续...

链表音乐播放器管理器

1. 人工添加歌曲
2. 显示播放列表
3. 删除歌曲 (按标题)
4. 播放歌曲 (按标题)
5. 导出歌单
6. 随机播放歌曲 (非必做)
7. 在指定位置添加歌曲 (非必做)
8. 清空播放列表 (非必做)
0. 退出程序

请选择操作 (0-8): 0

感谢使用链表音乐播放器管理器!

播放列表已清空

Process returned 0 (0x0) execution time : 264.191 s

Press any key to continue.



实验内容

实验12 音乐播放器 II

请在实验11的基础上，使用**双向链表**来存储和管理歌曲。实现以下功能：

- 从文件读取歌曲信息，并建立歌单链表
- 用户添加或删除歌曲信息
- 用户通过歌曲名查找并播放歌曲
- 显示歌单列表 (**正向+逆向**)
- 切换到**上/下**一首歌
- 将更新后的歌单信息写回到文件
- 根据歌名/作者/时长排序 (选做)

D:\code\c\2025\stu\test\bin\Debug\test.exe

```
正在从文件 song_list.txt 加载歌曲...
已添加歌曲: Beethoven - ForElise (ID: -1260584624)
已添加歌曲: Pachelbel - Canon (ID: -1260650160)
从文件成功加载 2 首歌曲
=== 双向链表音乐播放器管理器 ===
已加载 2 首示例歌曲
```

链表音乐播放器管理器

- ```
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
```

请选择操作 (0-12): \_



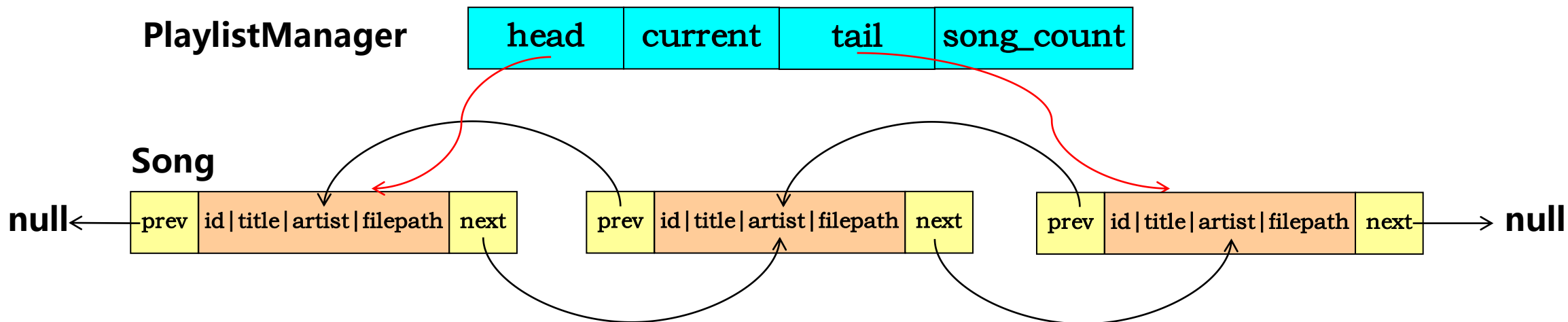
# 实验内容

//歌曲节点结构题

```
typedef struct Song{
 int id; //歌曲唯一标识
 char title[100]; //歌名
 char artist[50]; //歌手名
 char filepath[300]; //音频文件路径
 struct Song* prev; //指向上一首歌的指针
 struct Song* next; //指向下一首歌的指针
}Song;
```

// 播放列表管理器结构体

```
typedef struct PlaylistManager{
 Song* head; // 链表头
 Song* tail; // 链表尾
 Song* current; // 当前播放/操作的歌曲
 int song_count; // 歌曲总数
} PlaylistManager;
```



# 实验内容

//从文件读取歌曲信息构建双向链表

```
int load_songs_from_file(PlaylistManager* manager, const char* filename);
```

//在链表末尾添加歌曲

```
void add_song(PlaylistManager* manager, const char* title, const char* artist, const char* filepath);
```

//按标题删除歌曲

```
int delete_song_by_title(PlaylistManager* manager, const char* title);
```

//按标播放除歌曲

```
int play_song_by_title(PlaylistManager* manager, const char* title);
```

//显示播放列表（正向遍历）

```
void display_playlist(PlaylistManager* manager);
```

//显示播放列表（反向遍历）

```
void display_playlist_reverse(PlaylistManager* manager);
```

//将播放列表保存到文件

```
int export_playlist(PlaylistManager* manager, const char* filename);
```

//播放下一首歌曲

```
void next_song(PlaylistManager* manager);
```

//播放上一首歌曲

```
void previous_song(PlaylistManager* manager);
```

//随机播放歌曲（非必做）

```
int play_song_random(PlaylistManager* manager);
```

//在指定位置插入歌曲（非必做）

```
int insert_song_at(PlaylistManager* manager, int position, const char* title, const char* artist,
 const char* filepath);
```

//按照歌曲标题排序（非必做）

```
void sort_by_title(PlaylistManager* manager);
```



# 实验内容

D:\code\c\2025\stu\test\bin\Debug\test.exe

```
正在从文件 song_list.txt 加载歌曲...
已添加歌曲: Beethoven - ForElise (ID: 2081227088)
已添加歌曲: Pachelbel - Canon (ID: 2081161552)
从文件成功加载 2 首歌曲
=== 双向链表音乐播放器管理器 ===
已加载 2 首示例歌曲
```

链表音乐播放器管理器

- ```
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
```

请选择操作 (0-12): 4

播放列表 - 正向遍历 (2 首歌曲)

- ```
=====
1. ForElise - Beethoven
 文件路径: musics/ForElise.mp3
 前驱: NULL | 后继: Canon

2. Canon - Pachelbel
 文件路径: musics/Canon.mp3
 前驱: ForElise | 后继: NULL
=====
```

D:\code\c\2025\stu\test\bin\Debug\test.exe

按回车键继续...

链表音乐播放器管理器

- ```
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
```

请选择操作 (0-12): 5

? 播放列表 - 反向遍历 (2 首歌曲)

- ```
=====
2. Canon - Pachelbel
 文件路径: musics/Canon.mp3
 前驱: ForElise | 后继: NULL

1. ForElise - Beethoven
 文件路径: musics/ForElise.mp3
 前驱: NULL | 后继: Canon
=====
```

按回车键继续...



# 实验内容

```
链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 1
请输入歌曲标题: 大鱼
请输入作者: 周深
请输入歌曲路径: musics/大鱼.mp3

已添加歌曲: 周深 - 大鱼 (ID: 2081227568)

按回车键继续...

链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 4

播放列表 - 正向遍历 (3 首歌曲)
=====
1. ForElise - Beethoven
 文件路径: musics/ForElise.mp3
 前驱: NULL | 后继: Canon

2. Canon - Pachelbel
 文件路径: musics/Canon.mp3
 前驱: ForElise | 后继: 大鱼

3. 大鱼 - 周深
 文件路径: musics/大鱼.mp3
 前驱: Canon | 后继: NULL
=====
```

```
链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 3
请输入要播放的歌曲标题: Canon
Playing musics/Canon.mp3 ... 已播放标题为“Canon”的歌曲

按回车键继续...

链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 7
Playing musics/大鱼.mp3 ...
按回车键继续...
```

```
链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 8
Playing musics/Canon.mp3 ...
按回车键继续...

链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 8
Playing musics/ForElise.mp3 ...
按回车键继续...

链表音乐播放器管理器
=====
1. 添加歌曲
2. 删除歌曲 (按标题)
3. 播放歌曲 (按标题)
4. 显示播放列表 (正向)
5. 显示播放列表 (逆向)
6. 导出歌单
7. 切换到下一首歌
8. 切换到上一首歌
9. 随机播放歌曲(非必做)
10. 在指定位置添加歌曲(非必做)
11. 清空播放列表(非必做)
12. 按照歌曲名排序(非必做)
0. 退出程序
=====
请选择操作 (0-12): 8
已是第一首歌曲
```



# 实验报告

|              | 评分项     | 评分标准 |
|--------------|---------|------|
| 实验报告<br>(5分) | 系统设计    | 2分   |
|              | 函数设计    | 2分   |
|              | 系统结果与测试 | 1分   |

## ➤ 系统设计

- 描述选择使用的数据结构（例如链表、结构体等）
- 总体结构设计：main函数与各子函数之间的调用和返回关系，程序运行流程等

## ➤ 函数设计

- 函数接口定义：函数原型定义、实现的功能、参数及返回值含义
- 函数算法及函数逻辑流程的描述（每个函数需说明：实现思路 + 关键算法流程）

## ➤ 系统结果与测试

- 测试用例设计及测试结果截图，需体现正常值、边界值、异常值输入后的输出

## ➤ 问题与解决方法

## ➤ 课程总结与建议



# 流程图



**终端框（起止框）**：表示一个算法的开始或结束。里面的文字一般只是“开始”或“结束”



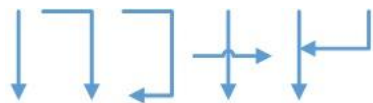
**输入、输出框**：表示一个算法输入和输出的信息。一般来说文字的开头要注明“输入”或“输出”。



**处理框（执行框）**：表示一个赋值、计算等操作。文字注明具体操作。



**判断框**：表示判断某条件是否成立。一般来说，它有两个分支，条件成立与否之后的流程在分支线处标明“是”“否”或“Y”“N”。



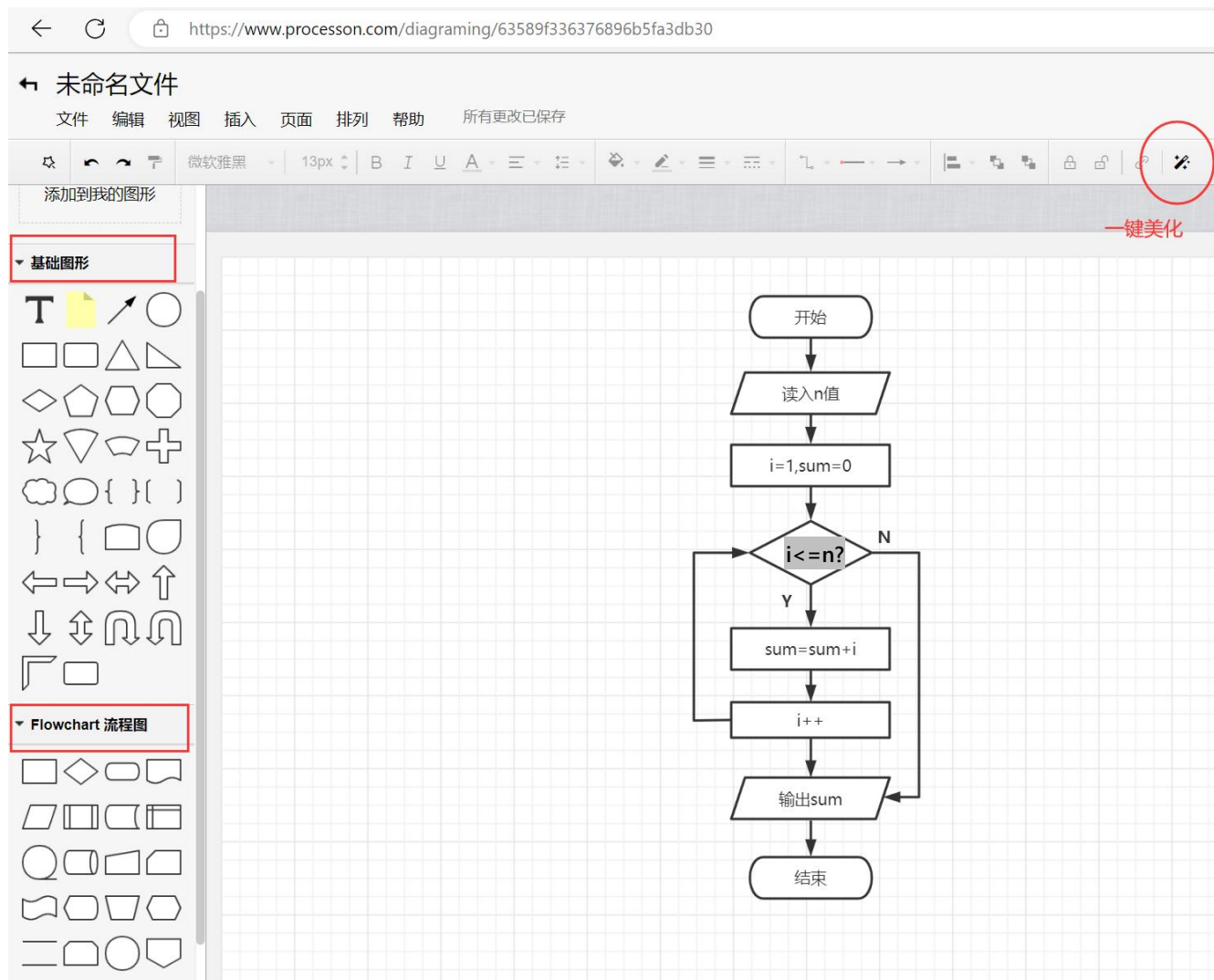
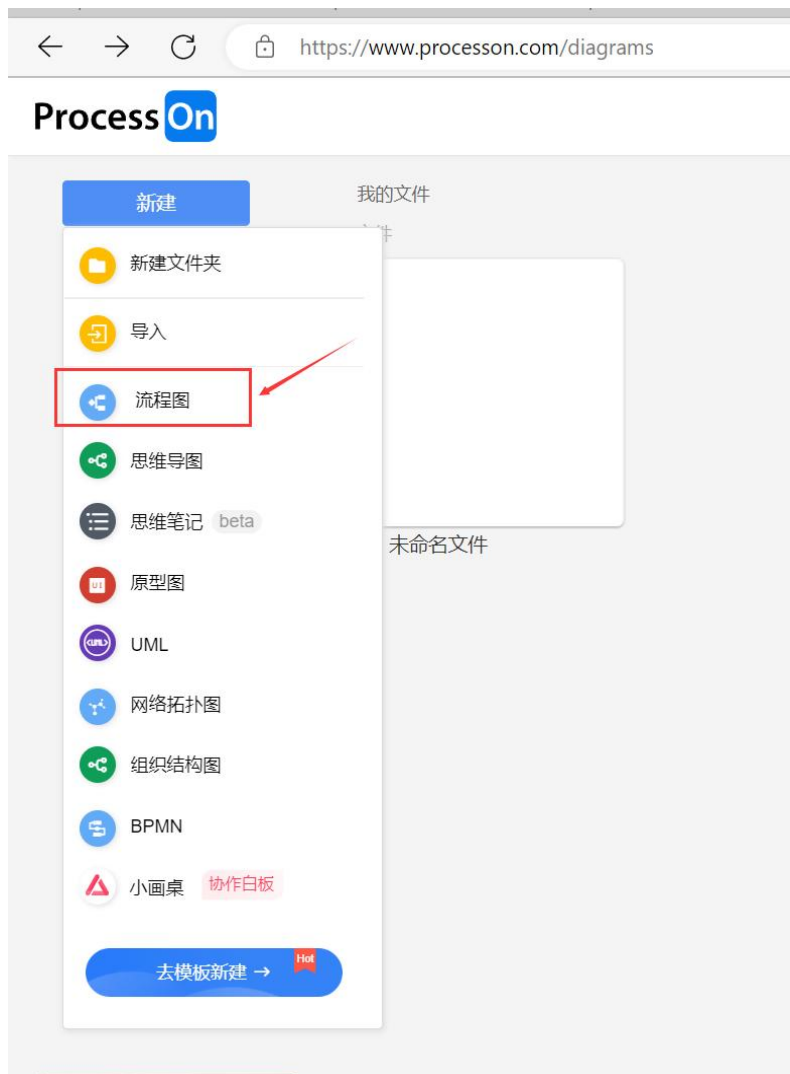
**流程线（指向线）**：流程图各符号之间以有向单向线连接。线一般要求横平竖直，可以有若干个90°的转弯。流程线尽量不要交叉，当两条流程线**不得已**而交叉时，将其中一条流程线的交叉处用圆弧隔开。

**可参考课本第五、六章**



# 流程图绘制工具

➤ ProcessOn: <https://www.processon.com/>





# 期末检查

- 请大家准备好前十个项目，使用教室机器检查
- 提前**确认每个项目都可以进行调试**
- 只检查一次，不进行补查，检查完的同学**不要提前离开教室**

|          | 检查项  | 评分标准                                                           |
|----------|------|----------------------------------------------------------------|
| 期末检查（5分） | 基本功能 | 1.编译通过，正确执行正常顺序流程，实现所有功能<br>2.对输入正常值、边界值、异常值可以进行判断和处理          |
|          | 程序设计 | 1.排序、查找算法<br>2.模块化程序设计                                         |
|          | 程序调试 | 1.正确创建工程文件,路径名命名正确<br>2.启动调试，单步或断点或打印语句调试程序，读懂调试结果并解释watch窗口内容 |
|          | 编码规范 | 1.代码整洁，层次清晰，易读，可维护<br>2.符合命名规范                                 |



# 作业提交

---

## 实验11 & 实验12 需提交到作业提交平台

网址: <http://10.249.12.98:8000>

### 提交内容:

- ① 完整的项目工程文件 (**实验11&12**) ;
  - ② 实验报告 (已提供报告模板, 提交PDF格式, **实验11 or 12**)
  - ③ 分别打包成zip压缩包后, 提交到收作业平台
  - ④ **DDL: 2025年12月31日, 截止之后不接受补交**
-

青 / 春 / 不 / 老 / 梦 / 想 / 永 / 在

FEEL THE MEANING OF THE TRIP

DREAM

MY DREAM WILL NEVER STOP

# 请同学们开始实验



哈爾濱工業大學(深圳)  
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

GO!  
TAKE YOU ON A TRIP

探索 从未停止